

minimised. This is achieved by having the results of the mapper phase partitioned on term IDs. For example, the term IDs can be partitioned into three partitions, namely:

Partition 1 -- containing those terms whose starting letter is between 'a-f',

Partition 2 -- containing those terms whose starting letter is between 'g-p', and

Partition 3 -- containing those terms whose starting letter is between 'q-z'.

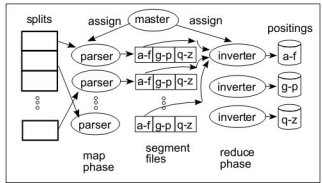
The intermediate results file at each mapper node is split into three segment files corresponding to each of these partitions. The master node assigns each partition to one reducer node, which processes all the segment files corresponding to that partition. This enables the reducer node to generate the final pos-list corresponding to the term IDs in that partition by merging the individual <term id, doc id> lists created at the end of the mapper phase. Figure 1 shows a typical example of distributed index construction.

Note that in the above description, we considered that each mapper node generates a partial list of <term id, doc id> corresponding to the split blocks it parses. However, we did not really answer the question of how the term ID is maintained unique for a given term across all mapper nodes. In other words, if a term *tl* is encountered, we need to associate an ID with it, and this ID should be the same across all mapper nodes. Otherwise this will cause an issue in merging the <term id, doc id> lists from different mapper nodes during the reduce phase. I will leave this as a question for our readers to come up with the answer.

So far, we have assumed that our document collection is fairly static and the building of an index is mostly an infrequent event. Under such an assumption, if there are modifications to the document collection, then the index can be rebuilt from scratch. But if we consider the 'World Wide Web' as our document collection, it is constantly changing. Hence rebuilding from scratch does not serve the purpose. Instead what we need is a way of rebuilding the index incrementally. One of the ways of achieving an incremental index construction is to have a main index and an auxiliary index. The former is the index built from the initial document collection. An auxiliary index is constructed only from those documents that have been modified since the last time of index building. When a query is received, the query terms are looked up against both the main as well as the auxiliary index, and the results are combined and returned to the user. Periodically, the auxiliary index is merged to the main index.

Now let us consider how to handle the case of documents getting deleted from the collection. We need to maintain a record of deleted document IDs in a list. Before the results are retrieved back to the user, they are filtered against the deleted documents list and only those documents that are currently in the collection are returned as valid results to the user.

As discussed earlier, the 'inverted index' is the major data structure for retrieval of documents based on the user query and,



Distributed Index Creation

hence, it needs to be very efficient. We would typically prefer that the entire 'inverted index' for a document collection session should fit in the main memory of the machine. But given the ever growing volume of the documents collected, we often encounter scenarios where the footprint of the inverted index cannot fit into the main memory. This results in slow look-ups on the inverted index, which in turn slows down the user queries. One way of mitigating this problem is to compress the inverted index and keep the compressed form in the main memory. We will go into more details about how an inverted index can be compressed and the issues related to that in the next column.

My 'must-read book' for this month

This month's book suggestion comes from one of our readers—Shilpa, and her recommendation is very appropriate to this month's column. She recommends an excellent article on information retrieval, titled, 'Modern Information Retrieval: A Brief Overview' by Amit Singhal, available at singhal.info/ieec2001.pdf. Note that Amit Singhal is a pioneering researcher in information retrieval and heads the 'search ranking' team at Google. Thank you, Shilpa, for sharing this link.

If you have a favorite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in this column. It would help many readers who want to improve their software skills.

If you have any favorite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=HYPERLINK> "http://www.linkedin.com/groups?home=&gid=2339182"&HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"gid=2339182